codeHive

Hash Tables

SPEED O(1): INSTANT ACCESS

The Final Word in Retrieval Performance

1. Why Hash Tables?

While Arrays and Linked Lists require **searching ($O(n)$)**, Hash Tables offer **direct access**. By using a **Hash Function**, the data itself determines its own address in memory.

AVERAGE SEARCH: O(1)

The 5-Step Logic:

1. Initialize an array of **Buckets**.
2. Run data through a **Hash Function**.
3. Convert the result to a valid array **Index**.
4. Store the data in that specific bucket.
5. Retrieve it instantly by re-running the same hash.

2. Mathematical Indexing

A hash function converts a value (like "Bob") into a numeric index. A common method is using Unicode points and the Modulo operator.

Example: Value "Bob"

`B(66) + o(111) + b(98) = 275`

`275 % 10 = index 5`

```
def hash_function(value, table_size):
    sum_chars = sum(ord(char) for char in value)
    return sum_chars % table_size
```

```
index = hash_function("Bob", 10) # Returns 5
```

3. Solving Collisions

What if "Lisa" and "Stuart" result in the same index? This is a **Collision**. We solve this using **Chaining**.

Chaining Method

Instead of storing one name, each bucket holds a small **Linked List** or **Array**. If multiple items hit the same index, they are simply appended to the chain.

```
# Implementing a Hash Set with Chaining
hash_set = [[] for _ in range(10)]

def add(name):
    idx = hash_function(name, 10)
    if name not in hash_set[idx]:
        hash_set[idx].append(name)

def contains(name):
    idx = hash_function(name, 10)
    return name in hash_set[idx]
```

4. Variety: Sets vs. Maps

Category	Hash Set	Hash Map
Content	Unique Keys Only	Key-Value Pairs
Goal	Check existence	Retrieve linked info
Example	Guest List check	Phonebook lookup

5. Real-World Power

Database Indexing: Most high-speed databases use Hash Tables to retrieve records instantly without scanning the entire hard drive.

Challenge: If a hash table has 1 million items, how many steps (on average) does it take to find one specific item?

Answer: Just 1 step (O(1)).